United International University

Department of Computer Science and Engineering

Bachelor of Science in Computer Science and Engineering

CSE 2217: Data Structure and Algorithms II

Final Examination: Fall 2025, Duration: 2 Hours, Total Marks: 40

Answer all questions. Marks are indicated on the right side of each question.

Any examinee found adopting unfair means will be expelled from the trimester/program as per UIU disciplinary rules.

---

1. a) The following table represents the parent array of a Disjoint Set Union (DSU) using the rooted-tree representation.

Index	0	1	2	3	4	5	6	7	8	9
Parent	0	1	1	2	1	0	5	5	7	7
Rank	5	5	-	-	-	-	-	-	-	-

Using path compression and union-by-rank, perform the following operations in order:

i. Perform Find-Set(4) and state the result and the resulting array. [1]

ii. Draw the disjoint-set forest represented by the array found in the previous step. [1]

iii. Redraw the forest after performing Union(3,8). [2]

iv. Redraw the forest after performing Union(5,4). [2]

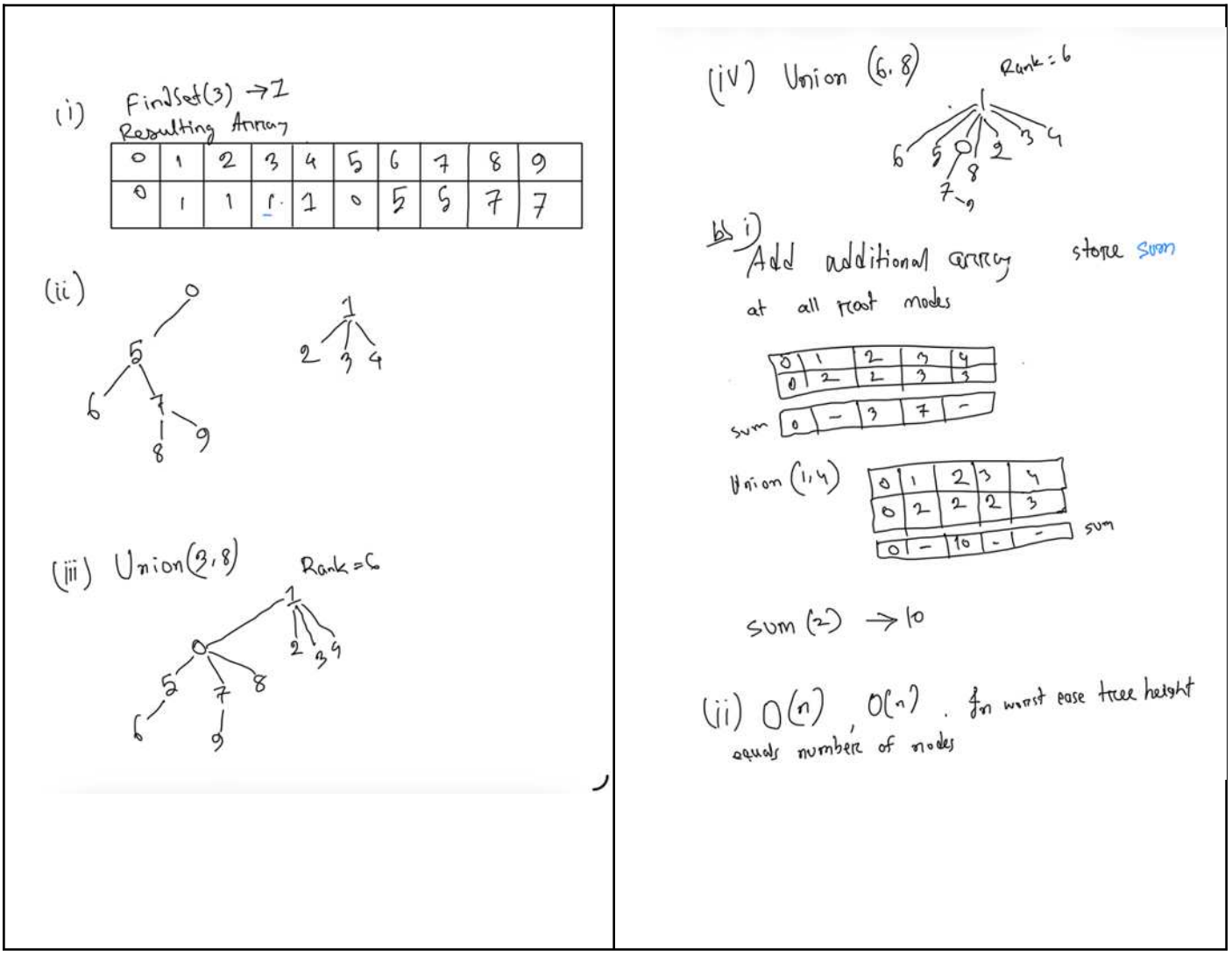
b) i. Modify the Disjoint Set Union (DSU) to support an additional operation, Sum(x), that returns the sum of the elements in the set containing x. Do not use path compression or union-by-rank. Clearly describe the modifications made to the DSU, including any additional data stored.

Given the following DSU represented by a parent array:

Index	0	1	2	3	4
Parent	0	2	2	3	3

Perform Union(1, 4) and then find Sum(2). [2]

ii. State the worst-case time complexity of Union(x, y) and Sum(x) with brief justification. [2]



2. a) A government authority plans to supply electricity from the national grid to five villages in a rural area. Each pair of villages can be connected by underground power cables, with the installation cost of a cable depending on the distance between them.

However, installing an excessive number of cables leads to electromagnetic interference, increased maintenance costs, and power loss. Therefore, the authority aims to design a network such that every village receives electricity either directly or indirectly. The total cable installation cost is minimized. No redundant cables are used to reduce interference.

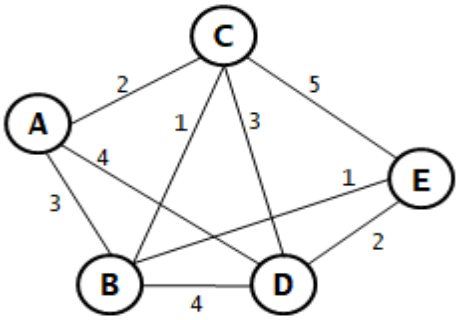


Figure 1 - Question 2(a)

The village network with the corresponding cable installation costs (in lakh) is given in Figure 1. Assuming the village network is a dense graph, use an appropriate algorithm with detailed simulation and determine: Which cable connections should be selected to connect all the villages with minimum total cost and the minimum total installation cost required. [6]

b) Under what condition is a graph guaranteed to have a unique minimum spanning tree? [1]

c) If an edge is added to or removed from a minimum spanning tree, will the minimum spanning tree property remain unchanged? Explain your answer with appropriate examples for each case. [3]

Solution and rubric:

Question-2(a):

- Dense graph → prim's algorithm [2 marks]
- Simulation Prim [2 marks]
- Chosen connections: (AC-2, BC-1, DE-2, BE-1) [1 mark]
- Total installation cost = 6 [1 mark]
  If a student uses Kruskal's algorithm, only 2 marks will be deducted, as s/he couldn't identify the appropriate algorithm.

Question-2(b):

If all edge weights are different → [1 mark]

Question-2(c):

(1.5 + 1.5)

Explanation with graphs.

Removing an edge: the tree will be disconnected. MST must connect all vertices.

Adding an edge: Will create a cycle. MST can't have any cycle.

3. a) Consider the undirected weighted graph with source node A (Figure 2). Apply Dijkstra's algorithm to find the shortest path from the source node to the rest of the nodes. [5]

b) If there are V vertices in a graph, why does the Bellman-Ford algorithm require V iterations to detect any negative cycle? [2]

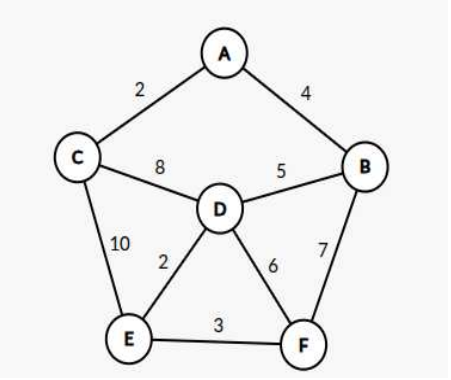


Figure 2 - Question 3(a)

c) The parent and distance arrays obtained after running Dijkstra's algorithm on a graph with source node A are given below.

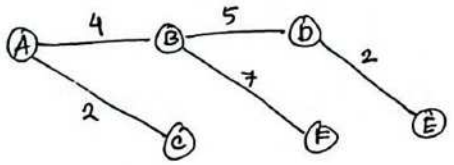
	A	B	C	D	E	F
parent	-	C	A	B	D	D
distance	0	3	2	8	10	13

Verify whether the algorithm was run correctly based on the values given in the parent and distance arrays. [3]

3.a)

π -	A	A	B	D	B
A	B	C	D	E	F
[0]	∞	∞	∞	∞	∞
	4	[2]	∞	∞	∞
	[4]		10	12	∞
			[9]	12	11
				[11]	11
					[11]

Shortest Path Tree:



Rubric:

- -2 if only edge weight are used to update (Prim)
- -2 if shortest path is not shown.

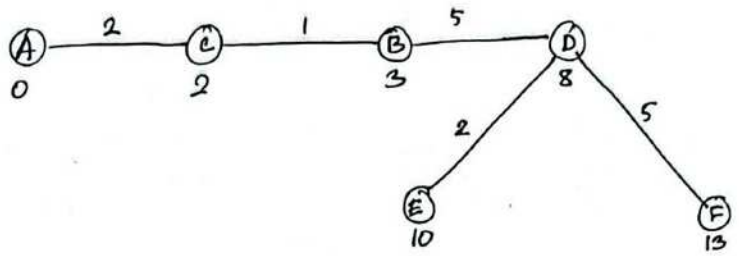
b) • A shortest path between any two nodes can have at most V-1 edges. Otherwise, it would have created a cycle and shortest paths cant contain any type of cycle.

- Each iteration of relaxation allows the shortest path information to propagate by one edge.
- Hence the Bellman Ford algorithm requires (V-1) iteration to find shortest paths and the Vth iteration is used to detect ~~exact~~ negative cycle.

Rubric:

- Marking will be done based on logical consistency

c) Shortest path tree drawn from the given corray



1. Assuming positive weighted graph, path cost is increasing along every path.

   So, from the given information it can be said that dijkstra was implemented successfully

2. If someone uses the graph given in the question, then the table is inconsistent.

Rubric:

- There is an ambiguity about whether the graph in the figure should be used. ~~At~~ Give full marks if answer is logically coherent.

4. Consider a hash table T of size m = 17 that uses open addressing with quadratic probing:

$$h(k, i) = (k \ mod \ 17 + i^2) \ mod \ 17, \quad i = 0, 1, 2, \ldots$$

The table is not empty. Initially, the following entries are already stored in T (all other slots are EMPTY):

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
T[i]		51			68			34			85	27			44		

a) Perform the following operations:
i. Insert the following keys in the given order into T: 102, 119, 136. Show the simulation for each entry. [3]
ii. Now delete the keys 34 and 85 by setting their table positions to NULL (EMPTY). [1]
iii. Perform Search(44) using the standard quadratic probing search procedure that stops when it encounters NULL. Show the probe sequence and the result. Then explain why the search can fail after deletion, and state one correct modification to the deletion/search strategy that prevents this problem. [2]

b) Suppose instead the hash table T had size m = 15 (non-prime) and still used quadratic probing:

$$h(k, i) = (k \ mod \ 15 + i^2) \ mod \ 15, \quad i = 0, 1, 2, \ldots$$

What kind of problem can quadratic probing face when m is not prime? Demonstrate the issue by showing the probe sequence of indices for k = 0 and explain what you observe.
Finally, does linear probing suffer from the same problem for m = 15? Briefly justify your answer. [4]

Solution (with Rubrics on the right)

a) i. Insert 102: $h(102, 0) = 0 \rightarrow$ index 0 is EMPTY → place 102 at 0 +1
Insert 119:
- $h(119, 0) = 0$(occupied: 102)
- $h(119, 1) = 1$(occupied: 51)
- $h(119, 2) = 4$(occupied: 68)
- $h(119, 3) = 9$(EMPTY) → place 119 at 9 +1

Insert 136:
- $h(136, 0) = 0$(occupied)
- $h(136, 1) = 1$(occupied)
- $h(136, 2) = 4$(occupied)
- $h(136, 3) = 9$(occupied: 119)
- $h(136, 4) = 16$(EMPTY) → place 136 at 16 +1

ii. Delete 34 and 85 by setting to NIL (EMPTY)
- 34 was at index 7 → set T[7] = NIL
- 85 was at index 10 → set T[10] = NIL

Table changes: index 7 and 10 become EMPTY. +1
iii. Start searching:
$h(44, 0) = 10 \rightarrow T[10] = NIL$(because 85 was deleted and set to NIL)
Search stops when it sees NIL, it terminates here. Returned result: Not found, even though the key 44 is actually stored at index 14. +1

Setting a deleted slot to NIL breaks the probing chain, so searches stop early and miss keys stored later in that chain. To prevent such issues, use a DELETED/tombstone marker for deletions so search continues probing past deleted slots (and insertion may reuse tombstones). Encountering that marker while searching,

- Search: treat marker as "keep probing"
- Insert: you may reuse marked slots +1

b) When m is not prime (e.g., m = 15), quadratic probing may not visit all table indices. The probe sequence can get stuck cycling through only a small subset of slots. As a result, insertion/search may fail even when empty slots exist elsewhere in the table (because those slots are never probed).
For k = 0:

$$h(0, i) = (0 + i^2) mod 15 = i^2 mod 15$$

Compute the probe indices:

- $i = 0$: $0^2 mod 15 = 0$
- $i = 1$: $1^2 mod 15 = 1$
- $i = 2$: 4
- $i = 3$: 9
- $i = 4$: $16 mod 15 = 1$← repeats (already saw 1)
- $i = 5$: $25 mod 15 = 10$
- $i = 6$: $36 mod 15 = 6$
- $i = 7$: $49 mod 15 = 4$← repeats (already saw 4)
- $i = 8$: $64 mod 15 = 4$← repeats again

…and it keeps repeating among the same residues.
So the set of indices reached is only: {0, 1, 4, 6, 9, 10}
That's 6 slots out of 15. Indices like 2,3,5,7,8,11,12,13,14 are never probed for $k = 0$. So, the sequence starts repeating early, meaning quadratic probing can "miss" many empty cells.
Does linear probing have the same problem for m = 15?
No. With linear probing,

$$h(k, i) = (k mod 15 + i) mod 15$$

the probe sequence advances by +1 mod 15, so it will eventually visit all 15 indices before repeating. (It still suffers from primary clustering, but it does not have this "can't reach many slots" coverage problem.)

Just simple explanation of quadratic probing not visiting all the indices: +1
Providing probe sequence to justify explanation: +2
Linear probing being able to work: +1